DHCP starvation and rogue DHCP

Follow-along: play the attacker, then the defender

You have three machines on one switched LAN (`101.0.0.0/24`):

Role	IP	Container
server	`101.0.0.1`	`101_L2_LAB_server`
attacker	`101.0.0.66`	`101_L2_LAB_attacker`
victim	leased over DHCP	`101_L2_LAB_victim`

They share one Open vSwitch switch (`101_L2_LAB_S1`). The switch interface inside each host is `101-S1`. The server runs a real ISC `dhcpd`: it leases addresses from `101.0.0.100` to `101.0.0.120` (21 addresses) and hands out `101.0.0.1`, itself, as the default gateway. The victim has no fixed address; it asks for one and trusts whatever it is told.

The attacker carries a stock kit; there is no bespoke exploit to run:

- `dhcp-starve`: sends full DHCP handshakes with invented MAC addresses until the server's pool is empty. You give it the interface.
- `dhcpd`: a second copy of the same ISC server the victim trusts. You write a short config for it and run it as the rogue server.
- `tcpdump`: read the DHCP exchange, and watch the traffic you divert.
- `udhcpc`: a DHCP client, used here to act as a machine joining the LAN.

IP forwarding is already on (it lets the attacker relay a victim's traffic once it is the gateway). Open a shell wherever a step says so with:

```
docker exec -it <container> bash
```

Assessment. This lab is graded on eighteen questions, marked inline below as **Question** *N*, each placed at the step whose output it asks about. Answer all eighteen in a separate document as you work through the steps. Every question is answered from what you see on screen at that step; none needs anything beyond the lab in front of you.

Part 1: the attack

1. **Confirm the honest baseline.** The victim already leased an address when the lab came up. On the victim:

   ```
   docker exec -it 101_L2_LAB_victim bash
   ip -4 addr show 101-S1       # an address in 101.0.0.100-120
   ip route show default        # default via 101.0.0.1 = the real server
   ```

   Write down the gateway. This is the value the attack will change.

   Question 1. The victim holds an address and a default gateway it never typed in. Name the four DHCP messages that assigned them, in order, and say which of those messages carried the default gateway.

2. **Recon: read an offer.** The attacker needs the server's address and the gateway it advertises, so it can impersonate that gateway. Watch one exchange. In one attacker shell, start a capture that decodes DHCP:

```
docker exec -it 101_L2_LAB_attacker bash
tcpdump -n -v -i 101-S1 'udp and port 68'
```

In a second attacker shell, trigger a single exchange (this grabs one address; the next step takes the rest):

```
docker exec -it 101_L2_LAB_attacker bash
dhcp-starve 101-S1 1
```

The capture prints the server's `Offer`. Read two fields from it: `Server-ID (54)` is the real server, and `Default-Gateway (3)` is the gateway you will later claim to be.

Question 2. In the `Offer` the capture shows, which field names the server the attacker must out-compete, and which field names the gateway it must impersonate? Give the value of each on this LAN.

Question 3. The victim accepts an offer without checking who sent it. What in the DHCP exchange lets the victim tell a legitimate server from a rogue one, and what does that tell you about the trust the protocol places in the first answer it hears?

3. **Starve the pool.** Now take every address. On the attacker:

```
dhcp-starve 101-S1        # runs until the server has no address left
```

Each fake client uses a fresh MAC and completes the whole handshake, so the server commits a lease to a client that will never use it. The tool prints each address as it locks it and stops when the offers run out.

Question 4. A lone `DISCOVER` does not hold an address for long. Which second message does the tool send to make the server commit the lease, and what happens to an address that is offered but never claimed?

Question 5. The tool reports the count of addresses it locked. Given the pool is `101.0.0.100` to `101.0.0.120`, how many addresses is the whole pool, and what does the real server now send back to a client that asks for one?

Question 6. One attacker drained the pool by inventing many MAC addresses. Explain why a new MAC counts as a new client to the server, and why that is what lets a single host consume the entire pool.

4. **See the denial.** A client the server already knows keeps its lease, so to see the damage you act as a machine the server has not met. DHCP identifies a client by its MAC, so give the victim a fresh MAC and ask for an address. On the victim:

```
ip link set 101-S1 down
ip link set 101-S1 address 02:aa:bb:cc:dd:ee
ip link set 101-S1 up
udhcpc -i 101-S1 -q -n -t 5 -T 1     # no offer comes back: the pool is empty
```

The client times out with no lease. A new machine on this LAN cannot get an address.

Question 7. The victim held a working lease a moment ago, yet with a new MAC it gets nothing. Explain why the server still answers the old MAC but not the new one.

5. **Answer as the rogue.** The real pool is empty, so a second server that still has addresses will win every new client. You run one on the attacker in three steps.

   First, write its config. This is a normal ISC `dhcpd` config; the only lie is the last line, the gateway it hands out. Run this on the attacker:

```
cat > /root/rogue.conf <<'EOF'
authoritative;                      # answer clients on this subnet
ping-check false;                   # offer at once, do not probe the address
    first
subnet 101.0.0.0 netmask 255.255.255.0 {
    range 101.0.0.150 101.0.0.200;  # a pool that does NOT overlap the real one
    option subnet-mask 255.255.255.0;
    option routers 101.0.0.66;      # the lie: the ATTACKER is the gateway
}
EOF
```

   The `cat > file <<'EOF' ... EOF` form is a here-document: it writes every line up to `EOF` into `/root/rogue.conf`. The range is `101.0.0.150-200` so the rogue never fights the real server for addresses; it only needs to answer, and to set `option routers` to itself.

   Second, create an empty lease database. `dhcpd` refuses to start without a lease file to write to, and starting empty means the rogue's pool begins full:

```
: > /root/rogue.leases
```

   `:` is the shell's do-nothing command; `: > file` runs it with its output redirected to `file`, which truncates the file to zero bytes (creating it if absent) without printing anything.

   Third, launch the rogue server on the LAN interface:

```
dhcpd -4 -q -cf /root/rogue.conf -lf /root/rogue.leases -pf /root/rogue.pid
    101-S1
```

   `-4` is IPv4 only, `-q` quiets the startup banner, `-cf` names the config, `-lf` the lease database, `-pf` a pid file, and the trailing `101-S1` binds it to the lab interface. It now leases from its own range and tells every client the gateway is `101.0.0.66`, you.

   Question 8. The rogue config changes one option from the real server's. Name the DHCP option and its value here, and state exactly what the client does with that value when it accepts the lease.

   Question 9. Starving the real pool first is what makes this reliable. If you had started the rogue *without* draining the pool, why would a new client's gateway be a coin toss, and what does draining the pool do to those odds?

6. **See the hijack.** Ask for an address again, still as the new client:

```
udhcpc -i 101-S1 -q -n -t 8 -T 1
```

```
ip route show default        # default now via 101.0.0.66 = the ATTACKER
```

The victim now routes through you and does not know it. This is the confirmed attack.

Question 10. The victim's default gateway is now the attacker. For which destinations does the victim's traffic pass through the attacker, and which destinations does it still reach directly without touching the attacker?

7. **Prove you are on-path.** Watch the victim's traffic to something off this subnet. In an attacker shell:

```
tcpdump -n -i 101-S1 'dst host 101.0.1.1'
```

On the victim, send to that off-subnet address:

```
ping -c3 101.0.1.1
```

The attacker's capture shows the victim's packets arriving at you, because you are its gateway.

Question 11. The capture catches the victim's packets to `101.0.1.1` but not its packets to the server `101.0.0.1`. Explain the difference in terms of the routing decision the victim makes for each destination.

Part 2: the defence

Your job on the defender side is to stop the rogue taking over, and to stop the flood draining the pool, so the victim gets a legitimate lease no matter when it asks.

The switch is the one point every DHCP message crosses, so the fix goes there. DHCP snooping does two things here. It trusts DHCP server replies only from the port the real server sits on, so a rogue on any other port is silenced. And it binds each access port to the one client MAC that belongs on it and drops DHCP requests from any other source MAC, so the flood, which forges a new MAC for every fake client, is dropped at the switch and never reaches the server. You build both as OpenFlow rules on `br0`. Everything below runs on the switch, not on a host.

Open a shell on the switch and read what it forwards with now:

```
docker exec -it 101_L2_LAB_S1 bash
ovs-ofctl dump-flows br0        # stock image: no DHCP rules, everything floods
```

Find the port numbers. The server's port is the only trusted one; the victim's and attacker's are access ports:

```
ovs-vsctl get Interface 101-server   ofport   # the TRUSTED port
ovs-vsctl get Interface 101-victim   ofport
ovs-vsctl get Interface 101-attacker ofport
```

Question 12. Of the three host ports, only the server's is trusted. What is true of the DHCP traffic a switch should ever see leaving the server's port that is not true of the victim's or attacker's port, and why does that make the server's port the right one to trust?

Block the rogue: trust server replies from one port only

A DHCP server reply travels from UDP port 67 to port 68. Permit that only from the trusted port, and drop it from anywhere else:

```
ovs-vsctl set bridge br0 protocols=OpenFlow10,OpenFlow13
ovs-ofctl -O OpenFlow13 add-flow br0
    "priority=300,in_port=<server-ofport>,udp,tp_src=67,tp_dst=68,actions=NORMAL"
ovs-ofctl -O OpenFlow13 add-flow br0
    "priority=200,udp,tp_src=67,tp_dst=68,actions=drop"
```

The rogue `dhcpd` sits on the attacker's port, so its offers are `67`→`68` frames on an untrusted port. They match the priority-200 drop and never reach a client.

Question 13. A client request travels `68`→`67`; a server reply travels `67`→`68`. Explain why matching on that direction, rather than on port 67 alone, lets the rule catch the rogue's replies without blocking any client's requests.

Question 14. The rogue's offer arrives on the attacker's port. State why it does not match the priority-300 permit rule, and which rule it lands on instead.

Stop the flood: bind each access port to one MAC

The trusted-port rule stops the rogue, but the attacker can still drain the pool by flooding the real server with requests. Look at how it does that: `dhcp-starve` forges a fresh, random source MAC for every fake client (a locally-administered `02:...` address), because the server keys each lease on the client's MAC and an unseen MAC reads as a new client. A starved port is therefore one port sourcing DHCP from dozens of different MACs in a few seconds.

A real access port has one client behind it, so it should only ever source DHCP from one MAC. Bind each access port to the MAC currently on it and drop DHCP requests (`68`→`67`) from any other source MAC. Read each client's MAC from its own host:

```
docker exec 101_L2_LAB_victim   cat /sys/class/net/101-S1/address   # the victim's
    MAC
docker exec 101_L2_LAB_attacker cat /sys/class/net/101-S1/address   # the
    attacker's real MAC
```

Then, on the switch, for each access port permit its bound MAC and drop every other source MAC's requests. Here is the attacker's port:

```
ovs-ofctl -O OpenFlow13 add-flow br0
    "priority=270,in_port=<attacker-ofport>,udp,tp_src=68,tp_dst=67,dl_src=<attacker-mac>,actions=N
ovs-ofctl -O OpenFlow13 add-flow br0
    "priority=260,in_port=<attacker-ofport>,udp,tp_src=68,tp_dst=67,actions=drop"
```

The priority-270 rule matches only the bound MAC, so it is more specific than the priority-260 drop and wins for the real client; every forged-MAC request falls through to the drop. Add the same pair of rules for the victim's port, with the victim's MAC. Then let non-DHCP traffic forward as a normal switch:

```
ovs-ofctl -O OpenFlow13 add-flow br0 "priority=10,actions=NORMAL"
```

Now the flood is dropped at the switch: the real server never sees a forged-MAC request, so it leases none of them and its pool stays full. This is the control that actually stops starvation, and it holds no matter when a client asks.

Question 15. `dhcp-starve` forges a new source MAC for every fake client. Explain why binding an access port to a single MAC and dropping requests from any other source MAC stops the flood outright, and why the real client on that port is unaffected.

Question 16. Both the permit (priority 270) and the drop (priority 260) match a forged request's port and its `68→67` direction; only the permit also names the bound source MAC. Explain, in terms of how a switch chooses between two matching flows, why the real client's request takes the permit and a forged-MAC request takes the drop.

Recover the server: clear the starved leases

The switch rules stop the attack from here on, but the damage from Part 1 remains: the flood already holds all 21 real addresses, so a new client still gets nothing from the real server. Refill the pool by clearing those leases.

Emptying the lease file alone does not do it. `dhcpd` reads its lease database into memory when it starts and works from that in-memory copy; truncating `/var/lib/dhcp/dhcpd.leases` while the server runs leaves the in-memory leases untouched, so the pool still reads as full and new clients are still refused. You have to restart the server. On the real server:

```
docker exec -it 101_L2_LAB_server bash
pkill -x dhcpd                                    # stop the server; its
    in-memory leases go with it
while pgrep -x dhcpd >/dev/null; do sleep 0.2; done # wait for it to release 101-S1
: > /var/lib/dhcp/dhcpd.leases                    # empty the on-disk lease
    database
dhcpd -4 -q -cf /etc/dhcp/dhcpd.conf -lf /var/lib/dhcp/dhcpd.leases -pf
    /var/run/dhcpd.pid 101-S1
pgrep -x dhcpd >/dev/null && echo "server up" || echo "dhcpd did not start: run
    this block again"
```

The `while pgrep` line is not optional: `dhcpd` holds `101-S1` until it exits, so a new server started before the old one lets go fails to bind and dies, leaving no server at all and no new leases. The catch is that `-q` silences `dhcpd`, that failure included, so a dead restart prints nothing and looks like it worked. The last line reads the result back: it prints `server up` when a `dhcpd` is running and `dhcpd did not start` when none is. If it reports failure, run the four lines again; the wait makes a second run safe. (`scripts/reset.sh` does the same restart with the same wait and check, but it also clears your switch rules; here you want the rules to stay, so restart the server by hand.) With a server up the real pool is full again, and port security keeps the flood from draining it again.

Question 17. Truncating `/var/lib/dhcp/dhcpd.leases` while the server is running does not free the pool, but restarting the server does. Explain where `dhcpd` keeps the leases it has handed out, and why that makes the file on its own the wrong thing to clear.

Re-run the attack under the defence

Run each half of the attack again and watch it fail. Take the flood first, on its own. Stop the rogue if you still have it running from Part 1 (so the count you read back reflects only the real server), then flood:

```
docker exec -it 101_L2_LAB_attacker bash
pkill -f rogue.conf 2>/dev/null   # leave only the flood running
dhcp-starve 101-S1                # reports: done: 0 address(es) locked
```

Every request the flood sends carries a forged MAC, so port security drops all of them at the switch: no offer comes back, so `dhcp-starve` gives up having locked zero addresses, and the real pool is untouched. (Leave the rogue running instead and the count is not zero, but those leases come from the rogue on the attacker's own host and reach no real client; the zero here is the honest measure of what the real server gave up, which is nothing.)

Now the other half: the rogue. Bring it back (its config from Part 1 is still at `/root/rogue.conf`) and ask for a lease as the victim, with the MAC the switch bound to its port:

```
: > /root/rogue.leases
dhcpd -4 -q -cf /root/rogue.conf -lf /root/rogue.leases -pf /root/rogue.pid 101-S1
```

```
# on the victim, its own MAC unchanged
ip addr flush dev 101-S1
udhcpc -i 101-S1 -q -n -t 5 -T 1
ip route show default        # via 101.0.0.1 = the REAL server, not 101.0.0.66
```

The rogue answers, but its offer is a 67→68 frame on an untrusted port, so the trusted-port rule drops it before it reaches the victim; the victim takes the real server's offer and its gateway stays `101.0.0.1`. On the switch, both drop counters have climbed, one per control:

```
docker exec -it 101_L2_LAB_S1 bash
ovs-ofctl -O OpenFlow13 dump-flows br0    # priority-260 (flood) and priority-200
    (rogue) n_packets
```

Unlike a rate limit, this holds regardless of timing: the flood never locks one of the server's addresses, so there is no window to miss. The victim leases whether it asks during the attack or long after it.

Question 18. On a production switch, DHCP snooping does more than drop each offending request: it acts against a port that keeps violating the bound-MAC rule, instead of dropping that port's forged packets one at a time for as long as it floods. Name that action, and say what it spares the switch from doing.

Checks: what "done" means

- **Attack works:** `dhcp-starve` drains the pool, a new-MAC client gets no lease from the real server, and once the rogue is running that client's `ip route show default` points at `101.0.0.66`. The attacker's `tcpdump` shows the victim's off-subnet traffic.
- **Defence holds:** with DHCP snooping and port security in place, the flood leases none of the real server's addresses (its forged-MAC requests are dropped), the victim leases from the real server with gateway **101.0.0.1** whether it asks during the flood or long after, and the `priority-200` (rogue) and `priority-260` (flood) drop counters both climb.

`scripts/status.sh` prints each host's addressing and the victim's leased gateway, so you can read the before and after without shelling in. `scripts/reset.sh` stops the attack, refills the

real pool, and re-leases the victim; `scripts/teardown.sh` removes the lab.

Safety. Everything here lives on one isolated Open vSwitch bridge with no route off the host. The flood and the rogue server only ever reach the lab's own containers.